Experiment-1.1

Student Name: Vaibhav Kumar
UID: 23BCS13386
Branch: B.E-C.S.E
Section/Group: 23KRG-2B
Semester: 5th
Date of Performance: 15/08/2025
Subject Name: DAA
Subject Code: 23CSH-301

1. **Aim:** Create c++ programs to manage product details, library systems, and student information using classes, inheritance, and abstraction.

2. **Objective:** To understand stacks.

3. **Input/Apparatus Used:** Stack are implemented using templates.

4. **Procedure:**
   Step1: Create stack.
   Step2: Check underflow and overflow condition.
   Step3: Increment top to store element in stack.
   Step4: Decrement top after removing element form stack.
   Step5: Check is stack empty or not.

5. **Code:**

```
#include <iostream>
#include <string>
using namespace std;

template <class T>
class Stack {
protected:
    int top;
    int capacity;
    T *arr;

public:
    Stack(int size = 5) {
        capacity = size;
        arr = new T[capacity];
        top = -1;
    }

    ~Stack() {
        delete[] arr;
    }
```

```
    bool isEmpty() {
        return top == -1;
    }

    bool isFull() {
        return top == capacity - 1;
    }

    void push(T data) {
        if (isFull()) {
            cout << "Stack Overflow! Cannot insert.\n";
            return;
        }
        arr[++top] = data;
        cout << "Inserted successfully.\n";
    }

    void pop() {
        if (isEmpty()) {
            cout << "Stack Underflow! Nothing to remove.\n";
            return;
        }
        cout << "Removed: ";
        arr[top--].display();
    }

    void displayAll() {
        if (isEmpty()) {
            cout << "Stack is empty.\n";
            return;
        }
        cout << "\n--- Stack Elements ---\n";
        for (int i = top; i >= 0; i--) {
            arr[i].display();
        }
    }
};

class Entity {
public:
    virtual void display() = 0;
};

class Product : public Entity {
    int id;
    string name;
    double price;

public:
    Product(int i = 0, string n = "", double p = 0.0) {
        id = i;
        name = n;
        price = p;
    }
```

```
    void display() override {
        cout << "Product ID: " << id << ", Name: " << name << ", Price: " << price << "\n";
    }
};

class Book : public Entity {
    int bookID;
    string title;
    string author;

public:
    Book(int id = 0, string t = "", string a = "") {
        bookID = id;
        title = t;
        author = a;
    }

    void display() override {
        cout << "Book ID: " << bookID << ", Title: " << title << ", Author: " << author << "\n";
    }
};


class Student : public Entity {
    int rollNo;
    string name;
    float marks;

public:
    Student(int r = 0, string n = "", float m = 0.0) {
        rollNo = r;
        name = n;
        marks = m;
    }
```

```
    void display() override {
        cout << "Roll No: " << rollNo << ", Name: " << name << ", Marks: " << marks << "\n";
    }
};
```

```cpp
int main() {
    Stack<Product> productStack(3);
    Stack<Book> bookStack(3);
    Stack<Student> studentStack(3);

    productStack.push(Product(1, "Laptop", 55000));
    productStack.push(Product(2, "Phone", 20000));
    productStack.displayAll();
    productStack.pop();

    bookStack.push(Book(101, "C++ Programming", "Bjarne Stroustrup"));
    bookStack.push(Book(102, "Data Structures", "Seymour Lipschutz"));
    bookStack.displayAll();

    studentStack.push(Student(1, "Vaibhav", 85.5));
    studentStack.push(Student(2, "Anjali", 90.0));
    studentStack.displayAll();

    return 0;
}
```

6.Output:

```
Inserted successfully.
Inserted successfully.

--- Stack Elements ---
Product ID: 2, Name: Phone, Price: 20000
Product ID: 1, Name: Laptop, Price: 55000
Removed: Product ID: 2, Name: Phone, Price: 20000
Inserted successfully.
Inserted successfully.

--- Stack Elements ---
Book ID: 102, Title: Data Structures, Author: Seymour Lipschutz
Book ID: 101, Title: C++ Programming, Author: Bjarne Stroustrup
Inserted successfully.
Inserted successfully.

--- Stack Elements ---
Roll No: 2, Name: Anjali, Marks: 90
Roll No: 1, Name: Vaibhav, Marks: 85.5
```